

Day 119 — Weekly Revision: Unsupervised ML, RL, Pipelines, MLflow and FastAPI

1. Day Number

Day 119

2. Topic Name

Weekly Revision — Days 113–118

Today you will revise:

- Unsupervised Machine Learning
- Reinforcement Learning basics
- scikit-learn Pipelines
- Kaggle workflow
- MLflow experiment tracking
- FastAPI basics

The goal is not to build a large project yet. It is to understand how these pieces can fit into a small practical ML workflow.

3. Connection

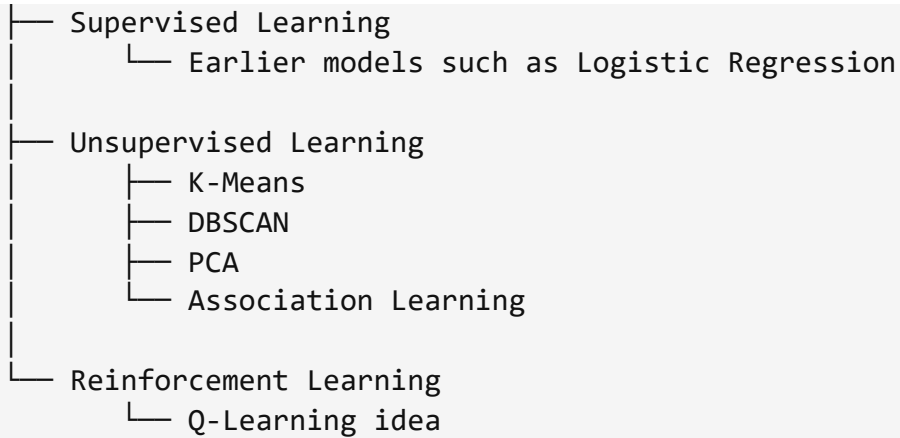
Earlier, you mainly studied **individual machine-learning ideas and algorithms**.

This week moved closer to a real workflow:

```
Raw Data
  ↓
Explore / Prepare Data
  ↓
ML Algorithm
  ↓
Pipeline
  ↓
Run Experiments
  ↓
Track Experiments with MLflow
  ↓
Choose / Save a Model
  ↓
Expose Predictions through an API
```

You also saw several different kinds of ML:

```
Machine Learning
|
```



4. Revision Summary of Days 113–118

Day 113 — K-Means Clustering

K-Means is an **unsupervised learning algorithm** used to divide similar observations into groups called **clusters**.

Example:

Suppose customers have:

```
age
income
spending_score
```

K-Means might discover:

```
Cluster 1 → low spending customers
Cluster 2 → medium spending customers
Cluster 3 → high spending customers
```

There is no predefined target such as:

```
customer_type = "high spender"
```

The algorithm discovers groups from the features.

Main idea:

```
Choose K cluster centers
  ↓
Assign points to nearest center
  ↓
Update centers
  ↓
Repeat
```

Day 114 — DBSCAN and PCA

DBSCAN

DBSCAN is another clustering algorithm.

Unlike K-Means, you normally do not tell it the exact number of clusters beforehand.

It looks for **dense regions of points**.

It can also identify some observations as **noise/outliers**.

Conceptually:

```
dense group → cluster  
dense group → cluster  
isolated point → noise
```

DBSCAN can be useful when clusters have unusual shapes.

PCA

Principal Component Analysis is mainly a dimensionality-reduction technique.

Suppose your data has:

```
10 features
```

PCA might represent much of the important variation using:

```
2 or 3 components
```

Think:

```
many related features  
    ↓  
PCA  
    ↓  
fewer summary dimensions
```

At this level, remember:

PCA tries to reduce the number of dimensions while preserving useful variation in the data.

Day 115 — Association Learning

Association learning discovers relationships between items.

A classic example is shopping baskets.

Suppose many customers buy:

bread + butter

You may discover a relationship such as:

bread → butter

This does **not** automatically mean bread causes people to buy butter.

It means the items often appear together in the observed data.

Common concepts include:

support
confidence
association rule

Day 116 — Reinforcement Learning and Q-Learning Idea

Reinforcement Learning is different from ordinary supervised learning.

An **agent** interacts with an **environment**.

```
Agent
  ↓ chooses
Action
  ↓
Environment
  ↓ returns
Reward + New State
```

Important terms:

Agent
State
Action
Reward
Environment
Policy

Q-Learning idea

Q-Learning tries to learn how useful different actions are in different states.

Conceptually:

$Q(\text{state}, \text{action})$

means:

How useful might this action be when I am in this state?

For example:

```
State: robot is in room A
Action: move right
Q-value: 8
```

A larger Q-value may indicate that the action has historically led to better future rewards.

You do not need advanced Q-Learning mathematics yet.

Day 117 — scikit-learn Pipeline and Kaggle Workflow

A **Pipeline** connects multiple ML steps together.

Without a Pipeline:

```
scale data
encode data
train model
predict
```

With a Pipeline:

```
Pipeline
├ preprocessing
└ model
```

Conceptually:

```
Pipeline([
    ("preprocessing", ...),
    ("model", ...)
])
```

The important benefit is consistency.

The same transformations can be applied during:

```
training
validation
prediction
```

Kaggle workflow

A beginner Kaggle workflow might look like:

```
Read problem
  ↓
Load dataset
  ↓
Inspect data
  ↓
Clean / preprocess
  ↓
Split data
  ↓
Create baseline model
  ↓
Evaluate
  ↓
Try improvements
  ↓
Generate predictions
```

Kaggle is useful practice because it makes you connect separate ML skills into one workflow.

Day 118 — FastAPI + MLflow Concepts

You started connecting ML experiments with a possible application interface.

FastAPI

FastAPI allows Python programs to create web APIs.

A future ML API might look conceptually like:

```
Client
  ↓
POST /predict
  ↓
FastAPI
  ↓
ML Pipeline / Model
  ↓
Prediction
  ↓
JSON Response
```

Example request:

```
{
  "age": 31,
  "income": 65000
}
```

Possible response:

```
{  
  "prediction": 1  
}
```

For now, the important idea is the flow rather than building the complete API.

Additional Day 118 MLflow Concepts

MLflow introduced another important layer:

```
experiment tracking
```

Instead of training many models and trying to remember what happened, MLflow lets you record the details of each experiment.

Example:

```
Experiment: Customer Classification
```

```
Run A
```

```
model = Logistic Regression
```

```
C = 1
```

```
F1 = 0.79
```

```
Run B
```

```
model = Random Forest
```

```
n_estimators = 100
```

```
F1 = 0.84
```

```
Run C
```

```
model = Random Forest
```

```
n_estimators = 200
```

```
F1 = 0.85
```

Now you can compare experiments systematically.

5. Important Topics

The main topics from this week are:

Topic	Main purpose
K-Means	Divide similar observations into K clusters
DBSCAN	Discover dense clusters and possible noise
PCA	Reduce the number of dimensions

Topic	Main purpose
Association Learning	Discover items/events that commonly occur together
Reinforcement Learning	Learn behavior through actions and rewards
Q-Learning	Learn useful state-action values
Pipeline	Combine preprocessing and model steps
Kaggle workflow	Practice an end-to-end ML problem
MLflow	Track and compare ML experiments
FastAPI	Provide a web API around Python/ML logic

6. Foundational Notes

Unsupervised learning does not require a target label

Supervised learning usually has:

```
features → target
```

For example:

```
age + income → will_buy
```

Clustering might have only:

```
age
income
spending_score
```

and try to discover structure.

Clusters are not automatically meaningful categories

Suppose K-Means creates:

```
Cluster 0
Cluster 1
Cluster 2
```

Those numbers do not automatically mean:

```
bad customer
average customer
good customer
```

You must inspect the cluster characteristics.

Scaling can matter

Imagine two features:

```
age:      18-80
income: 20,000-500,000
```

Income is numerically much larger.

Distance-based algorithms such as K-Means may therefore be heavily influenced by income unless the features are appropriately scaled.

A Pipeline can help:

```
Scaler
  ↓
K-Means
```

Pipelines improve consistency

Instead of manually remembering:

```
scale training data
scale validation data
scale future API data
```

you can keep the steps together.

Conceptually:

```
raw data
  ↓
pipeline
  ├── preprocessing
  └── model
  ↓
output
```

MLflow does not train the model for you

MLflow mainly helps record what happened.

Think:

```
scikit-learn → performs ML
MLflow       → records ML experiments
FastAPI      → exposes application functionality
```

A model metric without context can be misleading

Suppose two runs show:

```
Run A accuracy = 92%  
Run B accuracy = 90%
```

That does not automatically mean Run A is better.

You may need to consider:

```
precision  
recall  
F1  
class imbalance  
train/test split  
preprocessing
```

7. Easy Example

Suppose you have customer data:

Age	Income	Spending
21	25000	20
23	28000	24
46	70000	70
48	74000	76
35	45000	43
37	47000	46

You do not have a label saying what type of customer each person is.

You could ask:

Are there naturally occurring customer groups?

K-Means might be worth experimenting with.

A simple flow could be:

```
Customer Data  
↓  
StandardScaler  
↓  
K-Means
```

↓
Cluster Number

You could experiment with:

```
K = 2  
K = 3  
K = 4
```

and use MLflow to record those choices and evaluation information.

Later, an API could accept customer information and return a simple result.

For example:

```
POST /cluster
```

Request concept:

```
{  
  "age": 32,  
  "income": 50000,  
  "spending": 52  
}
```

Response concept:

```
{  
  "cluster": 1  
}
```

We are **not implementing this API today**.

8. Revision Problem Statement

You are given a small customer dataset containing:

```
age  
income  
spending_score
```

There is no predefined customer category.

Your task is to explain:

1. Whether clustering might be useful.
2. Which clustering method you could initially experiment with.
3. Why scaling could matter.
4. How you could create a simple scikit-learn Pipeline.

5. How MLflow could track several deliberately different experiments.
6. Which parameters and metrics you might record.
7. How you would keep the comparisons fair.
8. How a future FastAPI endpoint could accept customer information.
9. What simple JSON result that endpoint might return.

Do not implement the final API.

9. Concepts Used

This revision problem combines:

```
unsupervised learning
clustering
features
distance
scaling
Pipeline
experiments
runs
parameters
metrics
artifacts
model logging
reproducibility
JSON
HTTP endpoint
FastAPI
```

10. Thought Process

When approaching a small ML application, think in layers.

Step 1 — Understand the problem

Ask:

```
Do I have labels?
```

If no label exists and you are trying to discover groups:

```
clustering may be appropriate
```

Step 2 — Inspect the features

For example:

```
age
income
spending_score
```

Ask:

```
Are they numeric?
Are values missing?
Are the scales very different?
```

Step 3 — Choose a simple baseline

Do not immediately try many complicated ideas.

Start with something small:

```
StandardScaler
  ↓
K-Means
```

Step 4 — Create the Pipeline

Keep preprocessing and the algorithm together.

Conceptually:

```
Pipeline
|
|— StandardScaler
|
|— K-Means
```

Step 5 — Create deliberately different experiments

Instead of changing random things, create understandable runs.

For example:

```
Run 1
K = 2

Run 2
K = 3

Run 3
K = 4
```

Or, in a classification problem:

```
Run 1
Logistic Regression
class_weight = None

Run 2
Logistic Regression
class_weight = balanced

Run 3
Random Forest
n_estimators = 100
```

Step 6 — Track experiments with MLflow

Record:

```
what changed
what model was used
what preprocessing was used
what metric resulted
```

Step 7 — Compare runs fairly

Keep important parts consistent.

For example:

```
same dataset
same train/test split
same metric definitions
```

Otherwise you may accidentally compare two experiments that were tested under different conditions.

Step 8 — Think about the future API

Only after you understand the ML workflow should you think:

```
Input JSON
  ↓
FastAPI endpoint
  ↓
Pipeline
  ↓
Result
  ↓
Output JSON
```

11. Beginner-Friendly Pseudocode

```
LOAD small dataset

INSPECT dataset

SELECT useful features

IF features use very different numerical scales:
    include scaling

CREATE preprocessing + model Pipeline

CREATE MLflow experiment

FOR each deliberately chosen model/parameter setup:

    START MLflow run

    GIVE run a meaningful name

    LOG important parameters

    TRAIN Pipeline

    CALCULATE appropriate evaluation information

    LOG metrics

    LOG trained model

    LOG useful artifacts if needed

    END run

COMPARE runs in MLflow UI

SELECT promising experiment for further study

DESIGN FastAPI endpoint:

    RECEIVE JSON input

    VALIDATE required fields

    SEND values through trained Pipeline

    CREATE simple result
```

RETURN JSON

DO NOT build final API yet

12. Suggested Solving Approach

Use four simple layers.

Layer 1 — Conceptual ML

Decide:

What kind of problem is this?

For unlabeled groups:

clustering

Layer 2 — scikit-learn Pipeline

Combine:

preprocessing
+
model

Example concept:

StandardScaler → K-Means

or for a supervised experiment:

StandardScaler → LogisticRegression

Layer 3 — MLflow Experiment Tracking

For every important experiment:

start run
log parameters
train
evaluate
log metrics
log model/artifacts
end run

Then compare those runs.

Layer 4 — API Flow

Design:

```
JSON request
  ↓
FastAPI
  ↓
trained Pipeline
  ↓
result
  ↓
JSON response
```

Keep these layers separate in your mind.

13. MLflow Revision

A. Experiment and Run

An **experiment** is a collection of related ML attempts.

A **run** is one particular attempt.

Example:

```
Experiment:
Customer Churn Models

├─ Run 1: Logistic Regression
├─ Run 2: Logistic Regression balanced
└─ Run 3: Random Forest
```

Think:

```
Experiment = folder of ML attempts
Run        = one attempt
```

B. Parameter vs Metric

This distinction is very important.

Parameter

A parameter describes a choice you made.

Examples:

```
C = 1.0
n_estimators = 100
```

```
max_depth = 5
class_weight = balanced
K = 3
```

Think:

Parameter = what did I configure?

Metric

A metric describes the measured result.

Examples:

```
accuracy = 0.88
precision = 0.82
recall = 0.91
F1 = 0.86
```

Think:

Metric = how did the experiment perform?

C. Comparing Different Models

MLflow can help compare experiments such as:

```
Run A
model = LogisticRegression

Run B
model = DecisionTree

Run C
model = RandomForest
```

Each run could record:

```
model type
important hyperparameters
accuracy
precision
recall
F1
```

Then you can view them side-by-side.

D. Tracking Hyperparameters

Hyperparameters are settings chosen before or during model configuration.

Examples:

```
Logistic Regression:  
C
```

```
Random Forest:  
n_estimators  
max_depth
```

```
K-Means:  
n_clusters
```

You might log:

```
n_clusters = 3
```

so later you know exactly which configuration produced that run.

E. Class Weights

Consider:

```
950 normal examples  
50 rare examples
```

This is an imbalanced classification dataset.

Some algorithms allow something conceptually like:

```
class_weight="balanced"
```

You could create:

```
Run 1  
class_weight = None  
  
Run 2  
class_weight = balanced
```

and compare appropriate metrics.

Class weights are relevant mainly to certain **supervised classification models**, not K-Means itself.

F. Accuracy, Precision, Recall and F1

Accuracy

```
correct predictions
-----
all predictions
```

Useful when classes are reasonably balanced, but potentially misleading when one class dominates.

Precision

Think:

When the model predicted positive, how often was it correct?

Recall

Think:

Of the actual positive examples, how many did the model find?

F1

F1 provides a balance between:

```
precision
+
recall
```

MLflow can record all of these for different runs.

Example:

```
Run A
accuracy = 0.94
precision = 0.70
recall = 0.42
F1 = 0.52

Run B
accuracy = 0.91
precision = 0.68
recall = 0.76
F1 = 0.72
```

The numbers let you inspect different aspects of performance instead of relying only on accuracy.

G. Logging Trained Models

MLflow can record the trained model associated with a run.

Conceptually:

```
Run
├── parameters
├── metrics
└── trained model
```

This helps preserve the connection between:

```
configuration
performance
model
```

H. Logging Artifacts

An **artifact** is a useful file produced by an experiment.

Examples:

```
confusion_matrix.png
predictions.csv
feature_summary.txt
evaluation_report.txt
```

Conceptually:

```
Run
├── Parameters
├── Metrics
├── Model
└── Artifacts
    ├── confusion_matrix.png
    └── predictions.csv
```

I. Autologging Idea

Normally, you might manually write many logging commands.

Conceptually:

```
log parameter
log another parameter
log metric
log model
```

MLflow can support **autologging**, which automatically records useful information for supported ML libraries.

Think:

```
manual logging:
"I choose what to record."

autologging:
"MLflow automatically records many common details."
```

As a beginner, it is useful to understand manual logging first so you know what is actually being tracked.

J. Run Names and Tags

Imagine seeing:

```
Run_938429
Run_109128
Run_552810
```

That is difficult to understand.

Meaningful run names could be:

```
logreg_baseline
logreg_balanced
random_forest_100
```

Tags can provide extra descriptive information.

For example:

```
purpose = baseline
dataset_version = cleaned_v1
preprocessing = scaled
```

They make experiments easier to organize.

K. Comparing Runs in the MLflow UI

The MLflow UI allows you to inspect runs and compare information such as:

```
Run Name
Model
Parameters
Accuracy
Precision
```

Recall
F1
Artifacts

Instead of trying to remember:

Was 0.84 F1 from the balanced model or the normal model?

you can inspect the experiment history.

L. Tracking Pipeline / Preprocessing Changes

Do not only track model changes.

Preprocessing changes are also experiments.

For example:

```
Run 1
scaling = no

Run 2
scaling = StandardScaler

Run 3
scaling = MinMaxScaler
```

Or:

```
Run 1
features = age,income

Run 2
features = age,income,spending
```

The preprocessing choice can affect the result significantly.

MLflow should help you remember **what changed**, not just which model was used.

M. Reproducibility Idea

Reproducibility means being able to understand and repeat an experiment later.

Imagine you get:

```
F1 = 0.87
```

but you forgot:

```
train/test split  
model settings  
preprocessing  
random state  
features
```

The number is much less useful.

A better experiment record might include:

```
model = LogisticRegression  
C = 1  
class_weight = balanced  
scaler = StandardScaler  
test_size = 0.2  
random_state = 42  
F1 = 0.87
```

You now have a clearer description of what produced the result.

N. Several Deliberately Different Runs

A good beginner experiment should have a reason for each run.

For example:

```
Run 1 – baseline  
Logistic Regression  
normal class weights  
  
Run 2 – imbalance experiment  
Logistic Regression  
balanced class weights  
  
Run 3 – different model  
Random Forest  
100 trees  
  
Run 4 – parameter experiment  
Random Forest  
200 trees
```

This is better than randomly changing five settings simultaneously.

You want to understand:

What did I change, and what happened afterward?

14. Easy Edge Cases

Edge case 1 — Empty dataset

If the dataset contains no rows:

```
clustering cannot meaningfully run
```

Check that data exists before training.

Edge case 2 — Missing values

Example:

```
age = 32  
income = missing
```

Your model or preprocessing step may not accept the missing value directly.

You need an appropriate cleaning strategy.

Edge case 3 — Features with very different scales

Example:

```
age = 25  
income = 300000
```

Distance calculations may be dominated by income.

Scaling may help.

Edge case 4 — K larger than the useful number of groups

Choosing:

```
K = 20
```

for a tiny dataset of only a few observations is unlikely to give a useful clustering result.

Edge case 5 — API receives a missing field

Expected:

```
{  
  "age": 30,  
  "income": 50000,  
  "spending": 50  
}
```

But the client sends:

```
{  
  "age": 30  
}
```

The future API should validate required inputs instead of silently making assumptions.

Edge case 6 — MLflow parameter forgotten

Suppose:

```
Run A:  
F1 = 0.81  
  
Run B:  
F1 = 0.86
```

But for Run B you forgot to log:

```
class_weight
```

Later you may not know why the result changed.

Important experiment settings should be recorded.

Edge case 7 — Unfair MLflow comparison

Suppose:

```
Run A → train/test split #1  
Run B → completely different train/test split
```

If the objective is to compare only two model configurations, differences in the split can make the comparison harder to interpret.

For a simple controlled comparison, keep the evaluation setup consistent.

15. Common Mistakes to Avoid

1. Treating cluster numbers as meaningful labels automatically

text Cluster 0 ≠ automatically bad customers

2. Forgetting scaling with distance-based algorithms.

3. Assuming K-Means is always the correct clustering algorithm.

4. Thinking PCA is a clustering algorithm.

PCA reduces dimensions; it does not itself create clusters.

5. Confusing reinforcement learning with supervised learning.

6. **Thinking Q-Learning means memorizing only the immediate reward.**
Its basic goal is to learn useful state-action values considering future reward.
 7. **Preprocessing training data one way and future prediction data another way.**
 8. **Changing many experimental variables at once.**
 9. **Recording metrics but forgetting important MLflow parameters.**
 10. **Using accuracy alone for every classification problem.**
 11. **Comparing MLflow runs that were evaluated using inconsistent procedures without noting the differences.**
 12. **Saving a model without remembering which preprocessing steps belong with it.**
 13. **Jumping directly to FastAPI before confirming the ML workflow works properly.**
 14. **Putting preprocessing logic in several different places instead of using a Pipeline where appropriate.**
-

16. Quick Self-Check Questions

Question 1

You have customer features but no customer category labels.

Would **clustering** or **classification** be the more natural first thing to investigate?

Question 2

Why might this Pipeline make sense?

```
StandardScaler
  ↓
K-Means
```

What problem is the first step trying to reduce?

Question 3

In MLflow, which is a **parameter** and which is a **metric**?

```
n_estimators = 100

F1 = 0.84
```

Question 4

Two MLflow runs use:

```
Run A:
Logistic Regression
class_weight = None
```

```
Run B:  
Logistic Regression  
class_weight = balanced
```

Why could tracking both `class_weight` and metrics such as precision, recall and F1 be useful?

Question 5

Put these into a sensible future workflow:

```
FastAPI  
MLflow  
Pipeline  
raw input  
JSON response  
model experiment
```

Think about which pieces belong to **training/experimentation** and which belong to **serving a result**.

17. Hint Only

For the revision exercise, start with this mental picture:

```
Dataset  
  ↓  
Understand whether labels exist  
  ↓  
Choose clustering if appropriate  
  ↓  
Build preprocessing + model Pipeline  
  ↓  
Create MLflow experiment  
  ↓  
Run several deliberately different configurations  
  ↓  
Log:  
  parameters  
  metrics  
  model  
  useful artifacts  
  ↓  
Compare runs fairly  
  ↓  
Choose one promising workflow  
  ↓  
Design only:
```

POST /predict or /cluster



JSON input



Pipeline



JSON output

For MLflow, keep asking yourself two questions:

What did I change? → parameter / configuration

What happened because
of that experiment? → metric / artifact

That distinction is the main idea to carry forward before eventually connecting a trained scikit-learn Pipeline to FastAPI.